Módulo 00

Fundamentos de Python para datos

Ciencia de Datos e IA Aplicada: de los datos a las decisiones

Semanas 0–1 · Caso práctico: *TiendaSur*

Qué vas a aprender

- Escribir Python: variables, listas, diccionarios, condicionales, bucles y funciones
- Operar con arrays de NumPy sin bucles
- Cargar, limpiar, agrupar y combinar tablas con pandas
- Elegir y crear el gráfico adecuado para cada pregunta

Caso: 5 tiendas, 1 año de ventas, 20.000 tickets.
¿Qué tiendas, categorías y meses ingresan más? ¿Funcionan las promociones?

0.1

Por qué Python para datos; Jupyter y Google Colab

Del Excel al código

Límite de la hoja de cálculo	Lo que aporta Python
Informe rehecho a mano cada mes	**Reproducible**: se vuelve a ejecutar
Lento con cientos de miles de filas	**Escala**: millones de filas
Una herramienta por tarea	**Un ecosistema**: limpieza, estadística, ML, IA

> Dos horas de consolidación cada lunes → dos minutos con "Ejecutar todo"

El ecosistema que usarás

Librería	Para qué	Analogía
NumPy	Cálculo numérico	La calculadora rápida
pandas	Tablas	Excel + SQL
matplotlib	Gráficos	El lienzo
seaborn	Gráficos estadísticos	Plantillas sobre matplotlib

Todo lo que viene después (scikit-learn, statsmodels, PyTorch...) se apoya en NumPy y pandas.

Qué es un notebook

- Documento con **celdas de texto** (Markdown) y **celdas de código**
- El resultado aparece justo debajo: número, tabla o gráfico
- Cuenta una historia: explicación → código → resultado → conclusión

Atajos: `Shift + Enter` ejecutar · `Esc + A/B` nueva celda · `Esc + M/Y` texto/código

El kernel recuerda... y manda el orden

```
[1] precio = 10          [3] precio = 20
[2] print(precio)  →  muestra 10 ... o 20 si ejecutaste [3] antes
```

- El kernel es el Python que ejecuta las celdas y **guarda las variables**
- Cuenta el **orden de ejecución** (`[n]`), no el orden en pantalla
- Reiniciar = olvidar todo
- Antes de entregar: **Reiniciar y ejecutar todo**

Opción A · Google Colab (sin instalar nada)

1. Entra en `colab.research.google.com`
2. *Archivo → Abrir cuaderno → GitHub* (o *Subir cuaderno*)
3. **Archivo → Guardar una copia en Drive**
4. Ejecuta la celda de `import`
5. Si algo falla: *Entorno de ejecución → Reiniciar sesión + Ejecutar todo*

Atención: La máquina es temporal: los ficheros subidos se borran. No subas datos confidenciales.

Opción B · En tu ordenador

```
python -m venv .venv                    # entorno aislado
source .venv/bin/activate               # Windows: .venv\Scripts\activate
pip install -r requirements.txt
jupyter lab
```

- Alternativa sencilla: **Anaconda** (Navigator)
- Editor: **VS Code** + extensiones *Python* y *Jupyter* → *Select Kernel* `.venv`

Comprueba tu instalación

```
import sys, numpy as np, pandas as pd
print(sys.version)
print("NumPy", np.__version__, "| pandas", pd.__version__)
```

Error	Causa habitual
`NameError`	Celda anterior sin ejecutar / kernel reiniciado
`ModuleNotFoundError`	Librería en otro entorno distinto del kernel

Ideas clave · 0.1

- Python = reproducibilidad + escala + un solo ecosistema
- Notebook: texto + código + resultados; el kernel guarda el estado
- Colab: cero instalación (guarda copia en Drive)
- Local: entorno virtual o Anaconda
- **Reinicia y ejecuta todo** antes de entregar

0.2

Tipos, variables y operadores

Variables: etiquetas para valores

```
precio = 19.99            # float
unidades = 3              # int
producto = "Café"         # str
en_oferta = True          # bool
```

- `=` **asigna**; no significa "es igual"
- *snake_case* y nombres que expliquen: `ventas_semana`, no `x`
- Distingue mayúsculas; mejor sin tildes

Los tipos básicos

Tipo	Ejemplo	Uso en datos
`int`	`20_000`	Conteos, ids
`float`	`19.99`	Importes, %
`str`	`"Sevilla"`	Categorías
`bool`	`True`	Indicadores
`None`	`None`	Sin valor

Conversión: `int("3")` · `float("19.99")` · `str(450)`

Operadores aritméticos

Operador	Ejemplo	Resultado
`/`	`7 / 2`	`3.5` (siempre decimal)
`//`	`7 // 2`	`3`
`%`	`7 % 2`	`1`
`**`	`1.05 ** 3`	`1.157625`

Ante la duda: **paréntesis**

Ejemplo: precio final

$$p_{\text{final}} = p\,(1 - d)\,(1 + t)$$

```
p, d, t = 25.0, 0.15, 0.21
p_final = p * (1 - d) * (1 + t)
print(f"Precio final: {p_final:.2f} €")   # 25.71 €
```

Escribir la regla en código obliga a ser explícito

Cuidado con los decimales

```
0.1 + 0.2               # 0.30000000000000004
0.1 + 0.2 == 0.3        # False
import math
math.isclose(0.1 + 0.2, 0.3)   # True
```

- Para **mostrar**: `round(x, 2)` o `f"{x:.2f}"`
- Para **comparar**: tolerancia, nunca `==`

Comparaciones y lógica

```
es_caro = precio > 40
aplica = es_caro and not en_oferta
```

- `==` `!=` `<` `<=` `>` `>=` devuelven `bool`
- `and`, `or`, `not`
- `True` **vale 1**: sumar booleanos = contar; su media = proporción

$$\text{media}(\text{promocion}) \approx 0{,}20 \Rightarrow 20\,\%\text{ de tickets en promoción}$$

Texto, f-strings y errores típicos

```
print(f"{unidades} x {producto} = {total:.2f} €")   # 3 x Café = 59.97 €
" Sevilla ".strip()   ·   "19,99".replace(",", ".")
```

Error	Causa
`SyntaxError` en `if x = 10`	`=` en lugar de `==`
`TypeError` en `"Total: " + 59.97`	Texto + número
`ValueError` en `float("19,99")`	Coma decimal

Ideas clave · 0.2

- `=` asigna, `==` compara
- Tipos: `int`, `float`, `str`, `bool`, `None`; los ficheros traen texto
- `/` da decimal; redondea al mostrar, compara con tolerancia
- `True` = 1 → la media de una condición es una proporción
- f-strings para mostrar resultados

0.3

Listas, diccionarios y comprensiones

Listas: secuencias ordenadas

```
ventas_semana = [120, 135, 98, 160, 210, 250, 190]   # lun → dom
len(ventas_semana)                                    # 7
sum(ventas_semana) / len(ventas_semana)               # 166.14
ventas_semana.append(175)                             # son mutables
```

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

Índices y rebanadas

```
valor   120  135   98  160  210  250  190
índice    0    1    2    3    4    5    6
         -7   -6   -5   -4   -3   -2   -1
```

- `ventas_semana[0]` → 120 · `ventas_semana[-1]` → 190
- `ventas_semana[0:5]` → lunes a viernes (**el final no se incluye**)
- `ventas_semana[5:]` → fin de semana

Diccionarios: clave → valor

```
tienda = {"nombre": "Centro", "ciudad": "Sevilla", "m2": 450}
tienda["ciudad"]                          # 'Sevilla'
tienda.get("telefono", "sin dato")    # sin KeyError
for clave, valor in tienda.items():
    print(clave, valor)
```

Tabla de consulta (un BUSCARV):

```
categorias = {"Alimentación": 8, "Bebidas": 5, ...}
```

De estructuras a tablas

Estructura	Equivale a
Lista de diccionarios	Cada diccionario = una **fila**
Diccionario de listas	Cada clave = una **columna**

```
{"tienda": ["Centro", "Norte"], "m2": [450, 300]}
```

→ es exactamente cómo se crea un DataFrame (lección 0.6)

Comprensiones

$$[\ \text{expresión}\ \ \textbf{for}\ x\ \textbf{in}\ \text{colección}\ \ \textbf{if}\ \text{condición}\ ]$$

```
precios = [10, 25, 40]
[p * 1.21 for p in precios]              # [12.1, 30.25, 48.4]
[p for p in precios if p > 20]           # [25, 40]
{p: p * 0.9 for p in precios}            # diccionario
```

Si no cabe en una línea → bucle normal

Ejemplo: días que superan el objetivo

```
dias = ["Lun", "Mar", "Mié", "Jue", "Vie", "Sáb", "Dom"]
[d for d, v in zip(dias, ventas_semana) if v > 150]
# ['Jue', 'Vie', 'Sáb', 'Dom']
```

Errores frecuentes

Síntoma	Causa
`IndexError` con `lista[7]`	Índices de 0 a 6
`KeyError: 'Alimentacion'`	Falta la tilde
Cambia `a` al tocar `b`	`b = a` es un alias → `a.copy()`
Resultados raros	Modificar la lista mientras se recorre

Ideas clave · 0.3

- Listas: ordenadas, índices desde 0, `[a:b]` excluye `b`
- Diccionarios: clave → valor, tablas de consulta
- Lista de diccionarios / diccionario de listas = tabla
- Comprensiones para transformar y filtrar con claridad

0.4

Condicionales, bucles y funciones

Condicionales

```
if ingreso > 115:
    tipo = "grande"
elif ingreso > 40:
    tipo = "mediano"
else:
    tipo = "pequeño"
```

- **Dos puntos** y **sangría de 4 espacios**: obligatorios
- Se ejecuta solo el **primer** bloque verdadero → ordena de más a menos restrictivo

Ejemplo: comisiones por tramos

$$\text{comisión}(v) = \begin{cases} 0{,}02\,v & v \le 10\,000 \\ 200 + 0{,}04\,(v - 10\,000) & v > 10\,000 \end{cases}$$

Ventas	Comisión
8.000 €	160 €
10.000 €	200 €
15.000 €	400 €

Bucles `for`

```
total = 0
for v in ventas_semana:
    total += v
```

Herramienta	Uso
`range(n)`	0, 1, …, n−1
`enumerate(lista)`	posición y valor
`zip(a, b)`	dos listas en paralelo
`dic.items()`	pares clave, valor
`break` / `continue`	salir / saltar vuelta

Anatomía de una función

```
def aplicar_descuento(precio, porcentaje=10):
    """Devuelve el precio tras aplicar un descuento en %."""
    return precio * (1 - porcentaje / 100)

aplicar_descuento(40)        # 36.0
aplicar_descuento(40, 25)    # 30.0
```

`def` · parámetros · valor por defecto · docstring · `return`

La regla de negocio, en una función

```
def comision(ventas, umbral=10_000, tasa_baja=0.02, tasa_alta=0.04):
    """Comisión mensual por tramos."""
    if ventas <= umbral:
        return tasa_baja * ventas
    return tasa_baja * umbral + tasa_alta * (ventas - umbral)

[comision(v) for v in [8_000, 10_000, 15_000]]   # [160.0, 200.0, 400.0]
```

- Se cambia en **un solo sitio** · se **prueba** con casos conocidos

`lambda` y un aviso importante

```
lambda d: d["ingreso"] / d["m2"]     # función anónima de una línea
```

- La verás en `.assign(...)` (sección 5 del notebook)

Atención: Con tablas de datos **no recorras filas con `for`**:
usa operaciones vectorizadas (NumPy, pandas)

Errores frecuentes

Síntoma	Causa
`IndentationError`	Sangría inconsistente
La función devuelve `None`	Falta `return` (`print` ≠ `return`)
`<function ...>` en vez del resultado	Faltan los paréntesis al llamar
Ningún ticket es "grande"	`elif` en mal orden

Ideas clave · 0.4

- Sangría y dos puntos definen los bloques
- `for` + `enumerate`, `zip`, `.items()`
- Funciones = reglas de negocio reutilizables y comprobables
- `print` muestra, `return` devuelve
- Con datos tabulares: vectoriza

0.5

NumPy: arrays y vectorización

Las listas no son vectores

```
[1, 2, 3] * 2          # [1, 2, 3, 1, 2, 3]   ← repite
[1, 2] + [3, 4]        # [1, 2, 3, 4]         ← concatena

import numpy as np
np.array([1, 2, 3]) * 2    # array([2, 4, 6])  ← multiplica
```

El `ndarray` y la vectorización

```
v = np.array([120, 135, 98, 160, 210, 250, 190])
v * 1.21           # IVA a toda la semana
v / v.sum()        # cuota de cada día
v.shape, v.dtype   # (7,), int64
```

- Elementos **del mismo tipo**, guardados de forma compacta
- La operación se aplica a todo el array en código compilado

¿Cuánto más rápido?

Multiplicar 1 millón de números por 2	**Tiempo aprox.**
Comprensión de listas	decenas de ms
NumPy `a * 2`	< 1 ms

≈ 50–100 veces más rápido (depende del equipo)

... y el código se parece a la fórmula

Agregaciones y `ddof`

$$s = \sqrt{\frac{1}{n - \mathrm{ddof}} \sum_{i=1}^{n} (x_i - \bar{x})^2}$$

	Por defecto	Semana del notebook
`v.std()` (NumPy)	`ddof=0`	49,7
`pd.Series(v).std()`	`ddof=1`	53,7

`axis=0` → por columnas · `axis=1` → por filas

Máscaras booleanas

```
v > v.mean()                # [F, F, F, F, T, T, T]
(v > v.mean()).sum()        # 3   → contar
v[v > v.mean()]             # [210, 250, 190] → filtrar
(v > v.mean()).mean()       # 0.43 → proporción
v[(v > 100) & (v < 200)] # & | ~ con paréntesis
```

`np.where(cond, si, no)` = un "SI()" vectorizado

Crecimiento porcentual

$$g_t = \frac{v_t - v_{t-1}}{v_{t-1}} \times 100$$

```
(np.diff(v) / v[:-1] * 100).round(1)
# [ 12.5 -27.4  63.3  31.2  19.  -24. ]
```

`v[:-1]`: todos menos el último → ambos lados con 6 valores

Aleatorios reproducibles

```
rng = np.random.default_rng(42)            # semilla
rng.choice(["A", "B"], 5, p=[0.3, 0.7])    # elegir con probabilidades
rng.poisson(2, 5)                          # conteos
rng.normal(1, 0.1, 5)                      # ruido
```

- Misma semilla → mismos datos para todos
- Así se simula un año de TiendaSur (sección 3)

Errores frecuentes

Síntoma	Causa
`lista * 2` repite	Usar listas para cálculos
`ValueError: truth value ... ambiguous`	`and` / `or` con arrays
Resultado absurdo en `v > 100 & v < 200`	Faltan paréntesis
"Mi desviación no coincide"	`ddof` distinto
`operands could not be broadcast`	Tamaños incompatibles

Ideas clave · 0.5

- Arrays homogéneos + vectorización = rápido y legible
- Máscaras: contar, filtrar, proporciones
- `&`, `|`, `~` con paréntesis
- NumPy `ddof=0`, pandas `ddof=1`
- Semilla para reproducibilidad

0.6

pandas I: Series, DataFrame, selección y filtrado

Series y DataFrame

```
          Series                          DataFrame
índice  valor           índice  tienda   ciudad   m2
  0     450               0     Centro   Sevilla  450
  1     300               1     Norte    Sevilla  300
  2     380               2     Sur      Cádiz    380
```

- **Series**: valores + índice (una columna)
- **DataFrame**: columnas (Series) que comparten índice; cada una con su tipo

Cargar datos

```
pd.read_csv("ventas.csv")
pd.read_csv("ventas.csv", sep=";", decimal=",",
            encoding="latin-1", parse_dates=["fecha"])
pd.read_excel("ventas.xlsx", sheet_name="2025")
```

Atención: Exportaciones de Excel en español: `;` como separador y coma decimal

Primer vistazo (siempre)

Instrucción	Responde a
`head()`	¿Se ha leído bien?
`shape`	¿Cuántas filas y columnas?
`info()`	¿Tipos? ¿Nulos?
`describe()`	¿Valores imposibles?
`value_counts()`	¿Categorías raras?

TiendaSur: 20.080 filas; `precio_unitario` con 19.930 no nulos

Seleccionar columnas

```
ventas["unidades"]                  # Series
ventas[["fecha", "unidades"]]       # DataFrame (lista de nombres)
```

- Doble corchete = **lista** de columnas
- Mejor `df["col"]` que `df.col`

`loc` frente a `iloc`

	`loc`	`iloc`
Selecciona por	**Etiquetas**	**Posición**
Rango `0:2`	Incluye el 2	Excluye el 2
Ejemplo	`df.loc[0, "tienda"]`	`df.iloc[-1]`

Atención: Tras `concat`, el índice tiene etiquetas repetidas: `ventas.loc[5]` puede devolver **2 filas** → `reset_index(drop=True)`

Filtrar filas

```
ventas[ventas["promocion"]]
ventas[(ventas["categoria"] == "Ropa") & (ventas["promocion"])]
ventas[ventas["tienda_id"].isin([1, 2])]
ventas.loc[ventas["unidades"] >= 5, ["fecha", "unidades"]]
ventas.query("unidades >= 5 and promocion")
```

Misma idea que las máscaras de NumPy

Columnas nuevas y orden

```
ventas["ingreso"] = ventas["unidades"] * ventas["precio_unitario"]
ventas.sort_values("ingreso", ascending=False).head(10)
```

- Vectorizado: 20.000 productos en una operación
- Un bucle fila a fila: más lento y más propenso a errores

Errores frecuentes

Síntoma	Solución
`truth value of a Series is ambiguous`	`&` o `|` con paréntesis
`KeyError: 'Categoria'`	Revisa `df.columns`
La asignación "no hace nada"	`df.loc[mask, "col"] = valor`
Una sola columna al leer CSV	`sep=";"`, `decimal=","`

Ideas clave · 0.6

- Series = columna con índice; DataFrame = tabla de Series
- Explora antes de analizar
- `loc` etiquetas · `iloc` posiciones
- Filtra con máscaras; modifica con `loc`
- Columnas nuevas siempre vectorizadas

0.7

pandas II: limpieza (nulos, duplicados, tipos, fechas)

Primero, diagnosticar

```
ventas.info()
ventas.isna().sum()           # 150 nulos en precio_unitario
ventas.duplicated().sum()     # 87 filas duplicadas
ventas.describe()
ventas["ciudad"].value_counts()
```

Datos sucios → conclusiones falsas con aspecto de verdaderas

Duplicados: ¿80 u 87?

- El notebook introduce **80** duplicados a propósito
- `duplicated()` encuentra **87**
- 7 son ventas distintas que coinciden en todo

> Sin **id de ticket** no se pueden distinguir. Pide siempre una clave única.

```
ventas.drop_duplicates()                       # filas idénticas
ventas.drop_duplicates(subset=["id_ticket"]) # por clave
```

Deduplica **antes** de imputar o agregar

Nulos: ¿por qué faltan?

- `mean()` ignora los `NaN`... pero `3 * NaN = NaN`
- Faltan **al azar** → casi cualquier método vale
- Faltan **por un motivo** ligado al valor → riesgo de **sesgo**

> Ejemplo: los productos caros se registran a mano y se olvidan más

Estrategias para los nulos

Estrategia	Riesgo
Eliminar filas (`dropna`)	Sesgo si no son aleatorios
Mediana global	Ignora diferencias entre grupos
Mediana del grupo (notebook)	Reduce la variabilidad
Indicador + imputación	Permite revisar después

```
ventas["precio_unitario"] = ventas["precio_unitario"].fillna(
    ventas.groupby("categoria")["precio_unitario"].transform("median"))
```

Nunca `fillna(0)` en un precio

Tipos de datos

```
pd.to_numeric(df["importe"].str.replace(",", "."), errors="coerce")
df["tienda_id"].astype(int)
df["categoria"].astype("category")
df["ciudad"].str.strip().str.title()     # "sevilla " → "Sevilla"
```

`errors="coerce"`: lo no convertible pasa a `NaN` → cuenta y revisa

Fechas

```
pd.to_datetime(df["fecha"], format="%d/%m/%Y")   # 03/04 = 3 de abril
```

`.dt`	Resultado
`.year`, `.month`	Año, mes
`.dayofweek`	0 = lunes
`.day_name()`	Nombre (inglés)
`.to_period("M")`	`2025-03`
`.dayofyear`	1–365

Resultado y errores frecuentes

TiendaSur tras limpiar: **19.993 filas, 0 nulos**

- Limpiar sin diagnosticar ni contar filas perdidas
- Imputar antes de deduplicar
- Fechas día/mes ambiguas
- `dropna()` sin `subset`
- No documentar las decisiones

Ideas clave · 0.7

- Diagnóstico → tratamiento → verificación
- Deduplica por clave única y antes de imputar
- Todo tratamiento de nulos tiene un riesgo: márcalo
- Tipos explícitos; fechas con formato y `.dt`

0.8

pandas III: `groupby`, tablas dinámicas y `merge`

Dividir – aplicar – combinar

```
 tienda ingreso        dividir          aplicar sum     combinar
 Centro   50     →  Centro: 50, 30  →   Centro  80  →  Centro  80
 Norte    20        Norte: 20, 40  →   Norte   60     Norte   60
 Centro   30
 Norte    40
```

```
ventas.groupby("tienda_id")["ingreso"].sum()
```

Varias métricas con nombre

```
ventas.groupby("categoria").agg(
    ingreso_total=("ingreso", "sum"),
    tickets=("ingreso", "size"),
    ticket_medio=("ingreso", "mean"),
)
```

`sum` · `mean` · `median` · `count` (no nulos) · `size` (filas) · `nunique`

`as_index=False` → la clave queda como columna

`agg` frente a `transform`

	Devuelve
`groupby(...).sum()`	**Una fila por grupo**
`groupby(...).transform("sum")`	**Una fila por registro** (total repetido)

$$\text{cuota}_{ij} = \frac{\text{ingreso}_{ij}}{\sum_k \text{ingreso}_{ik}} \times 100$$

```
t["cuota"] = 100 * t["ingreso"] / t.groupby("ciudad")["ingreso"].transform("sum")
```

Tablas dinámicas

```
pd.pivot_table(ventas_t, index="categoria", columns="tienda",
               values="ingreso", aggfunc="sum", margins=True)
pd.crosstab(a, b, normalize="index")      # proporciones
```

categoría	Centro	…	Aeropuerto
Electrónica	107.278	…	54.753

`merge`: unir por una clave

```
ventas_t = ventas.merge(tiendas, on="tienda_id", how="left",
                        validate="many_to_one")
```

`how`	Conserva
`inner`	Claves en ambas
`left`	Todas las de la izquierda
`outer`	Todas

Comprueba `len()` antes y después (19.993)

Total frente a ratio

```
(ventas.groupby("tienda_id", as_index=False)["ingreso"].sum()
   .merge(tiendas, on="tienda_id")
   .assign(ingreso_m2=lambda d: d["ingreso"] / d["m2"])
   .sort_values("ingreso", ascending=False))
```

Tienda	Ingreso (€)	€/m²
Centro	287.486	639
Aeropuerto	143.831	**959**

Promociones: primer vistazo

promoción	unidades	precio	ingreso/ticket
No	3,01	16,09 €	48,80 €
Sí	3,84	13,58 €	51,78 €

- Más unidades, menor precio, ingreso ≈ +6 %
- ¿Real o ruido? ¿Efecto mezcla de categorías?
- → Módulos 01 y 04

Errores frecuentes

Síntoma	Causa
Más filas tras el merge	Claves duplicadas en la tabla de consulta
Menos filas tras el merge	`inner` o tipos de clave distintos
Media global errónea	Media de medias
`count` ≠ `size`	`count` ignora nulos

Ideas clave · 0.8

- `groupby` = dividir, aplicar, combinar
- `agg` reduce; `transform` difunde (cuotas, imputación)
- `pivot_table` / `crosstab` para resúmenes cruzados
- `merge` con `how`, `validate` y control de filas
- Total ≠ ratio: elige la métrica según la decisión

0.9

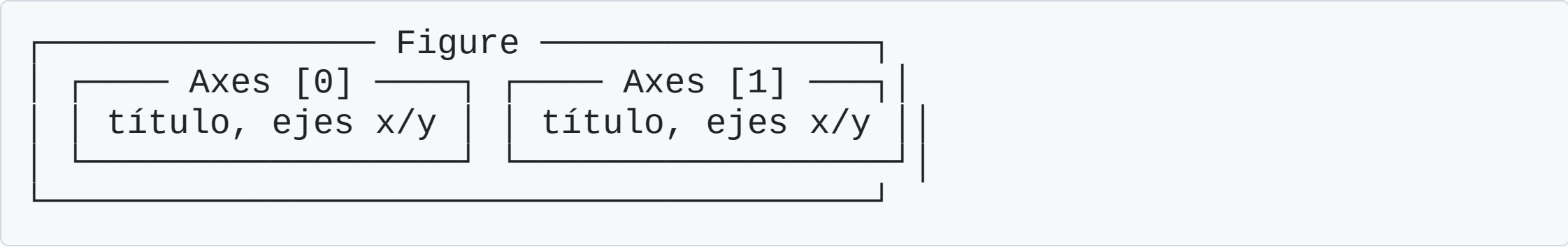
Visualización: qué gráfico usar para cada pregunta

De la pregunta al gráfico

Pregunta	Gráfico
Comparar categorías	Barras
Evolución temporal	Líneas
Distribución	Histograma / boxplot
Relación entre dos números	Dispersión
Composición	Barras apiladas al 100 %
Dos categorías cruzadas	Mapa de calor

Tartas: solo con 2–3 partes muy distintas

Anatomía en matplotlib



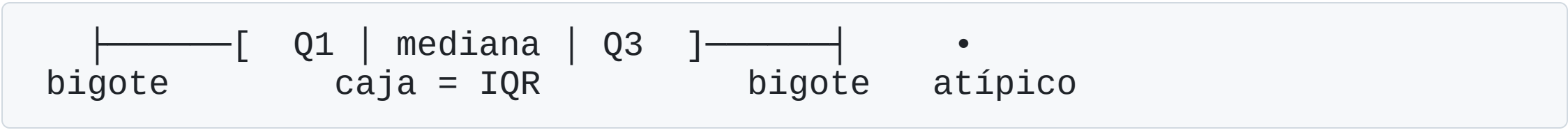

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
```

El código de la sección 6

```
mensual.plot(ax=axes[0], marker="o")
axes[0].set(title="Ingresos mensuales", ylabel="€")
sns.barplot(data=por_tienda, x="tienda", y="ingreso_m2",
            ax=axes[1], color="steelblue")
plt.tight_layout(); plt.show()
fig.savefig("ingresos.png", dpi=150, bbox_inches="tight")
```

`ax=` decide dónde se dibuja

El boxplot, en cinco números



$$\text{IQR} = Q_3 - Q_1 \qquad \text{bigotes hasta } Q_1 - 1{,}5\,\text{IQR} \text{ y } Q_3 + 1{,}5\,\text{IQR}$$

```
sns.boxplot(data=ventas, x="categoria", y="ingreso", hue="promocion")
```

Principios de un gráfico claro

1. Título = **conclusión**
2. Barras **ordenadas** y **desde cero**
3. Ejes con **unidades**
4. Un color neutro + un **color de acento**
5. Comparar lo comparable: **ratios** o **índices**
6. Fuera 3D y adornos

Ejemplo: ¿dónde ampliar Electrónica?

Métrica	Gana	Problema
Ingreso total de Electrónica	Centro	Solo refleja el tamaño
Ingreso de Electrónica por m²	Aeropuerto	Responde a la decisión de espacio

→ Barras ordenadas, ganadora resaltada, línea con la media de la cadena

Errores frecuentes

Error	Solución
`sns.barplot` muestra la **media**	Agrega antes o `estimator="sum"`
Líneas entre categorías	Barras
Eje truncado en barras	Empieza en 0
12 colores, 12 categorías	Agrupa en "Otros"
Sin título ni unidades	`ax.set(...)`

Ideas clave · 0.9

- La pregunta decide el gráfico
- Figure contiene Axes; dibuja con `ax=`
- Boxplot: mediana, cuartiles, IQR; `hue` para subgrupos
- Título con la conclusión, barras ordenadas desde 0
- La métrica debe responder a la decisión

¡A practicar!

Notebook del módulo · 5 ejercicios para Dirección
Código + resultado + **una frase de conclusión**

Próximo módulo: **01 · Estadística para la toma de decisiones**